Game Physics for Operation Code Rescue

Establishing and Maintaining Proper Chaos Physics Throughout the Experience

Operation Code Rescue Prepared with Claude

June 25, 2026

Contents

A technical chapter for a solo developer building a first-person survival-horror coding game on macOS / Apple Silicon.

0.1 1. Why This Chapter Exists

Operation Code Rescue asks the player to do two very different things at once: solve real coding puzzles at terminals, and survive a zombie-infested 465-city campaign. The survival-horror half of that promise lives or dies on **physics feel** — the weight of a thrown flare, the way a zombie crumples when shot, the crunch of a barricade, the heft of the Jeep across a ruined street. Today the project's physics are deliberately rudimentary. Combat is hitscan; there is no ragdoll; the Jeep rides on `UFloatingPawnMovement`; throwables simulate as inert spheres with no launch impulse; and only a handful of cover props are truly rigid-body simulated. This is a sensible *prototype* posture, but it is not commercial-release quality.

This chapter documents how to establish robust, performant, deterministic physics across the whole experience using **Chaos**, Unreal Engine 5.7's built-in physics solution, and — just as important — how to *maintain* that quality as the project grows. Chaos is Epic's lightweight, ground-up physics engine covering rigid-body dynamics, destruction (Geometry Collections), ragdoll, physical animation, vehicles, cloth, physics fields, and networked physics, all driven by a single solver [1]. Everything below is tailored to this project's actual classes (`AThrowableActor`, `ABarricadeActor`, `AJeepActor`, `ACodeZombieActor`, `APickupActor`, `ACodeRescueCharacter`) and to the constraints of a one-person team shipping on Metal.

> **Version note.** UE 5.7 is the shipping release as of mid-2026. Epic serves a single "latest" documentation channel, so several cited pages currently render a "5.8" label even though their content is unchanged from 5.7; where that is the case it is noted. Class names, settings, and defaults below are valid for 5.7.

0.2 2. Chaos Physics in UE 5.7: Solver, Substepping, and Determinism

0.2.1 2.1 The solver and why determinism matters

Chaos runs an iterative constraint solver: each tick it resolves collisions and joints over a configurable number of iterations, then integrates body state forward. In the Project Settings **Chaos Physics** section you can tune **Iterations**, **Collision Pair Iterations**, **Push Out Iterations**, **Joint Pair Iterations**, **Collision Max Push Out Velocity**, and **Collision Cull Distance**; the threading model is set by **Default Threading Model** and can be switched live with `p.Chaos.ThreadingModel` [2]. For a horror game, *consistency of feel* is the goal: a thrown grenade should land the same way every time, and a ragdoll should not jitter differently at 30 fps than at 120 fps.

The single most important lever for that consistency is the **Asynchronous Physics** mode, which "improves the determinism of the simulation and allows for predictable results every time the simulation runs" [1]. When enabled, the simulation runs on a parallel physics thread at a **fixed time step** decoupled from the variable game-frame rate, communicating with the game thread through synchronized buffers [3].

0.2.2 2.2 Substepping

With substepping enabled (**Edit > Project Settings > Physics > Substepping**), Unreal splits each frame's delta time into smaller fixed pieces and steps physics on each one, so a low or fluctuating frame rate does not degrade physics accuracy [4]. Four settings govern it: **Substepping**, **Substepping Async**, **Max Substep Delta Time**, and **Max Substeps** [4]. Epic's own worked example: if a full step takes 0.05 s and **Max Substep Delta Time** is 0.025, the step splits into two substeps; the count is capped by **Max Substeps**, and **Max Physics Delta Time** limits the longest permitted step [4]. "The smaller the max sub-step time the more stable your simulation will be but at a greater CPU cost," and the most visible payoff is reduced "ragdoll jitter and other complex physical assets" [4].

Recommended starting configuration for *Operation Code Rescue*:

1. Enable **Substepping**. Set **Max Substep Delta Time = 0.0166** (60 Hz minimum physics rate — a common convention; note Epic's documented example value is 0.025, so treat 1/60 as a project choice, not Epic guidance [4]).
2. Set **Max Substeps = 4** (so the solver can cover frames as slow as ~15 fps without exploding step counts).
3. Leave **Tick Physics Async** / **Substepping Async** *off* until the synchronous path is stable; both are flagged experimental, and async substepping is unsupported on mobile [2][4]. macOS desktop is fine, but validate it deliberately.
4. Set **Bounce Threshold Velocity 0.2 × gravity** (Epic's stated value for simulation stability) so small impacts don't chatter [2].

0.2.3 2.3 Async physics tick for gameplay code

When you need gameplay logic to run *in lockstep* with each physics substep (e.g. a future predictive grenade-trajectory preview), enable `bAsyncPhysicsTickEnabled` on the actor and handle the **Async Physics Tick** event, which provides a fixed **Delta Seconds** and accumulated **Sim Seconds** [5]. This is the deterministic path; ordinary `Tick()` is not. For most of this single-player project, the standard game tick is sufficient — reserve async tick for systems where reproducibility actually matters.

0.3 3. Collision Setup Done Right

0.3.1 3.1 The current state and its problem

Today the project sets collision almost entirely through built-in preset *strings*: `ACodeZombieActor` uses `"BlockAllDynamic"` on its capsule, `AThrowableActor` uses `"PhysicsActor"`, `APickupActor` uses `"OverlapAllDynamic"`, and several actors call `SetCollisionResponseToChannel(ECC_Visibility, ECR_Block)` ad hoc. This works, but it bundles every interaction into a few coarse buckets and forces line traces to share the generic `ECC_Visibility` channel — the same channel used by the elite Spitter's acid trace and the AI line-of-sight checks in `ACodeRescueAIController`. As the

game grows, coarse channels cause two failures: false positives (a trace meant for "can the zombie see the player" also hits decorative glass), and wasted query cost.

0.3.2 3.2 Presets, object channels, and trace channels

A **collision profile** is a preset bundling response settings (Block / Overlap / Ignore) for every channel, defined under **Project Settings > Engine > Collision** [6]. Best practice is to *create profiles and apply them* rather than hand-editing per-component responses, which prevents drift and simplifies maintenance [6]. Channels come in two flavors: **object channels** tag *what a thing is*; **trace channels** describe *a behavior you query for* (e.g. a weapon trace) [7]. UE allows up to 18 custom channels [7]; keep the default response **Ignore** and opt specific pairs into Block/Overlap to avoid silent performance loss [6].

0.3.3 3.3 Recommended channel scheme

Define these **custom object channels** in `DefaultEngine.ini` (via the Collision settings UI):

Channel	Purpose	Notable responses
`Player` (object)	The `ACodeRescueCharacter` capsule	Blocks World, Cover, Zombie; overlaps Pickup
`Zombie` (object)	`ACodeZombieActor` / `ABossZombieActor` capsules	Blocks World, Cover, Player; ignores other Zombies (perf)
`Cover` (object)	Barricades, destructible cover props	Blocks Player, Zombie, Projectile
`Pickup` (object)	Pickup/throwable trigger volumes	Overlap-only with Player

And these **custom trace channels**:

Trace channel	Used by	Replaces
`Weapon`	Hitscan shots, Spitter acid	`ECC_Visibility` for combat
`AISight`	`ACodeRescueAIController` LoS, companion checks	`ECC_Visibility` for perception
`Interaction`	Terminal/pickup focus traces	ad-hoc `ECC_Visibility`

Splitting `Weapon` from `AISight` lets glass and foliage block sight while staying transparent to bullets, and prevents the player's own body from spoiling a perception trace.

0.3.4 3.4 Complex vs. simple collision and performance

Every static mesh carries **simple** collision (convex hulls / primitives) and **complex** collision (per-polygon). Queries default to simple, which is cheap; per-poly complex traces are far more expensive and should be reserved for cases that need surface fidelity [8]. For *Operation Code Rescue*:

- Player capsule, zombie capsules, throwables, the Jeep body → **simple** collision only.
- Static city geometry the player walks on → simple collision generated to match silhouette; reserve complex collision for large architectural meshes where the player must stand on intricate ledges.
- **Weapon/AISight traces** → trace against **simple** collision wherever possible. Only trace complex when you need the precise surface (see §6 on physical materials — reading a per-body surface type requires a complex trace [9]).

0.4 4. Rigid-Body Dynamics for Gameplay Props

The project already does this correctly in places — `ACodeRescueGameMode` spawns simulated cover with concrete tuning: `SetSimulatePhysics(true)`, `SetLinearDamping(0.24f)`, `SetAngularDamping(0.36f)`, and `SetMassOverrideInKg(...)`. The goal is to generalize that discipline.

0.4.1 4.1 The core API

On any `UPrimitiveComponent` (typically a `UStaticMeshComponent`): `SetSimulatePhysics(true)` to enable rigid-body dynamics; `SetMassOverrideInKg(NAME_None, Kg, true)` to author mass directly; and `AddImpulse`, `AddForce`, `AddTorqueInRadians`, and `AddImpulseAtLocation` to drive motion [10]. An **impulse** is an instantaneous one-frame velocity change (use for hits and throws); a **force** accumulates over time (use for sustained pushes).

0.4.2 4.2 Damping and stability

Linear Damping and **Angular Damping** simulate drag [10]. Epic's reference points are precise: under normal gravity a Linear Damping of ~30 resists the initial gravity pull, ~100 stops a body that has had force applied, and an Angular Damping of ~100 "will almost immediately stop any angular motion" [10]. The project's existing cover values (0.24–0.42) are intentionally low so debris tumbles freely but settles — good defaults to copy.

0.4.3 4.3 Recommended starting values per prop class

Prop	Mass (kg)	Linear Damp	Angular Damp	Notes
Thrown flare/smoke (`AThrowableActor` Body)	0.6	0.05	0.10	Light, should bounce and roll
Barricade (`ABarricadeActor`, when knocked)	60	0.30	0.45	Heavy; resists shove
Light debris (planks, cans)	2–8	0.20	0.35	Matches existing cover values

Prop	Mass (kg)	Linear Damp	Angular Damp	Notes
Toppling cover (shelving)	40	0.25	0.40	Falls convincingly, then sleeps

0.4.4 4.4 Constraints and sleeping

For hinged props (a swinging door, a dangling sign) use a **Physics Constraint Component** (`UPhysicsConstraintComponent`) to join two bodies, configuring swing/twist/linear limits and soft-limit stiffness/damping [11]. Epic's stiffness reference: a Limit Stiffness of 50 barely affects motion, 5000 bounces back into the limit, 50000 fully deflects it [10]. To protect the frame budget, ensure idle props go to **sleep** (Chaos deactivates resting bodies); keep damping high enough that knocked props settle within a second or two rather than micro-jittering forever — exactly what the cover damping values above achieve.

0.5 5. Character Physics: Movement, Ragdoll, and Hit Reactions

0.5.1 5.1 Character Movement Component vs. physics movement

`ACodeRescueCharacter` and `ACodeZombieActor` are `ACharacter` subclasses driven by the **Character Movement Component (CMC)**. Crucially, CMC moves avatars "not using rigid body physics" — it is a *kinematic* system supporting walking, falling, jumping, swimming, and flying, paired with the capsule, and it can push physics objects without itself being simulated [12]. This is the right tool for both player and zombies: predictable, network-friendly, animation-driven locomotion with stepping and slope handling. Physics-based pawns (a simulating primitive moved by the solver) are reserved for bodies where authentic momentum matters more than authored motion — which is precisely the Jeep's case (§7). Do **not** convert zombies or the player to physics pawns.

0.5.2 5.2 Ragdoll on death — the headline upgrade

The current death path (`ACodeZombieActor`) plays a montage, disables collision, and destroys the actor after a delay. Replacing this with **ragdoll** is the single highest-impact physics improvement for horror feel.

A **Physics Asset** (`UPhysicsAsset`) defines the rigid bodies and constraints that make up a ragdoll for a skeletal mesh; only one Physics Asset is allowed per skeletal mesh, and any physics on a skeletal mesh *requires* it [13][14]. Author it in the **Physics Asset Editor** (formerly PhAT) — created automatically on FBX import with "Create Physics Asset" enabled, or from the Content Drawer on an existing mesh [13]. To ragdoll on death:

1. Ensure each zombie skeletal mesh has a sane Physics Asset (capsule/sphere bodies per limb, constrained at joints).
2. On death, set the mesh's collision profile so it interacts with the world but ignores the Pawn channel (preventing the corpse from fighting other capsules) [15].

3. Call `SetAllBodiesBelowSimulatePhysics(PelvisBone, true)` — this recursively activates simulation from a given bone down the chain [14].
4. Disable the CMC and capsule collision so the simulated bodies fall freely.

Because the project's Fab zombie packs ship as skeletal meshes, most already include or can quickly be given a Physics Asset. Budget ragdolls aggressively (see §9): cap simultaneous active ragdolls and let older corpses freeze or fade.

0.5.3 5.3 Physical Animation Component for hit reactions

For *living* zombies that flinch when shot — without going fully limp — use the **Physical Animation Component** (`UPhysicalAnimationComponent`) to blend physics over the playing animation. The workflow [16]:

1. Add the component; in BeginPlay call `SetSkeletalMeshComponent` to bind it to the mesh.
2. Call `ApplyPhysicalAnimationProfileBelow(BoneName, ProfileName, ...)` — passing e.g. `spine_01` so only the upper body is driven (passing `None` drives all bodies) [16].
3. Call `SetAllBodiesBelowSimulatePhysics(spine_01, true)` to begin simulating the targeted region [16].
4. Drive `SetAllBodiesBelowPhysicsBlendWeight` per tick: at 1.0 the bones are fully physics-driven; at 0.0 they return to keyframed animation [14]. Ramp quickly to ~0.6 on impact, then back to 0 over ~0.25 s.
5. Apply the kick with `AddImpulseAtLocation(ShotDirection * Force, HitLocation)` on the hit bone [17].

This is the bridge between today's static hitscan damage and AAA-grade reactive enemies, and it reuses the same Physics Asset as the death ragdoll.

0.6 6. Projectiles, Ballistics, and Surface-Specific Impacts

0.6.1 6.1 Hitscan vs. projectile

The project's combat — and the Spitter's acid — use **hitscan**: an instantaneous `LineTraceSingleByChannel(..., ECC_Visibility, ...)` reading `Hit.ImpactPoint` and `Hit.ImpactNormal` [18]. Hitscan is cheap, frame-rate-independent, and correct for fast bullets; keep it for the player's primary fire (but migrate it to the new `Weapon` trace channel from §3.3). Use **projectiles** only where travel time is a feature: thrown items, a future grenade launcher, or a slow acid glob.

0.6.2 6.2 ProjectileMovementComponent

For thrown and launched objects, add a **Projectile Movement Component** (`UProjectileMovementComponent`). Epic's own first-person tutorial uses these citable defaults: `InitialSpeed = 3000`, `MaxSpeed = 3000` (0 = no limit), `bRotationFollowsVelocity = true`, `bShouldBounce = true`, `Bounciness = 0.2` (the coefficient of restitution), `Friction = 0.8`, and a 5-second lifespan [18][19]. `ProjectileGravityScale` controls arc (0 = no gravity) [19]. The component fires **OnProjectileBounce** and **OnProjectileStop** delegates you can hook for effects [19].

Concrete fix for `AThrowableActor`: today the throwable is a simulated sphere with *no launch velocity and no explosion.* Two valid upgrades:

- *Minimal:* keep `SetSimulatePhysics(true)` and apply `AddImpulse(CameraForward * ThrowStrength, true)` at spawn so the flare actually arcs from the player's hand. With `bVelChange = true` the throw is mass-independent and easy to tune.
- *Preferred:* give it a `UProjectileMovementComponent` (InitialSpeed ~1200, ProjectileGravityScale 1.0, bShouldBounce true, Bounciness 0.3) for a predictable, art-directable arc that still bounces off walls.

0.6.3 6.3 Radial impulses for explosions

When a future grenade (or the smoke canister) bursts, apply force to everything nearby with **Add Radial Impulse** on a primitive component, or a `URadialForceComponent`. The node radiates an impulse from an **Origin** out to a **Radius**, with a **Strength** and **Falloff**; the **Vel Change** flag makes it mass-independent [20]. Starting values for a frag-style burst: Radius 450, Strength 90000, Falloff = linear, affecting `Cover` and `Zombie` channels. Pair it with ragdoll activation so nearby zombies are physically thrown.

0.6.4 6.4 Physical materials for surface fidelity

A `UPhysicalMaterial` carries **Friction**, **Restitution** (bounciness), and **Density** (g/cm^3), plus combine modes (default Average) [21]. **Surface Types** — `SurfaceType1…SurfaceTypeN`, with UE5 supporting many by default — are defined under **Project Settings > Physics > Physical Surfaces** in `DefaultEngine.ini` and read back from a trace's `Hit.PhysMaterial` [9][21]. This is how you select the right impact: a bullet hitting `SurfaceType_Concrete` spawns a dust puff and ricochet decal; `SurfaceType_Flesh` spawns blood and a wet hit sound; `SurfaceType_Metal` sparks. Note that reading the surface type from a skeletal mesh's Physics Asset body requires a **complex trace**, which returns the body's *Simple Collision Physical Material* [9]. Define a compact surface set early (Concrete, Metal, Wood, Glass, Flesh, Dirt) and key all impact VFX/SFX off it — this single system makes every shot in the game feel placed in the world.

0.7 7. Destruction: Breakable Cover and Environment

Survival-horror thrives on a world that comes apart. Chaos Destruction builds on the **Geometry Collection** (`UGeometryCollection`) asset, created from a static mesh and fractured in UE5's **Fracture Mode** [22]. Fracture methods include **Uniform** (Voronoi sites — e.g. 20 sites → 20 pieces), **Cluster**, **Radial**, **Planar**, **Slice**, and **Brick** [23]. A fixed **Random Seed** reproduces an identical fracture pattern (use this so destruction is consistent and testable); **Grout** sets the gap between pieces [23].

At simulation start Chaos builds a **connection graph** from each fractured piece's nearest neighbors, joining them with rigid constraints carrying **strain values**; a piece breaks when a collision or **Physics Field** impulse exceeds its connection limit [22]. **Damage Thresholds** drive breakage

directly — Epic's example uses thresholds of 200,000 and 500,000, where a field applying 400,000 of internal strain shatters the weaker collection but not the stronger [24]. **Anchor Fields** pin parts of a collection as static so a wall fractures but its base stays put [24].

Performance budgeting (critical for Apple Silicon). Real-time fracture of high piece counts is expensive. Mitigations: (1) use the **Cache System**, which records complex destruction once and replays it at runtime with minimal cost — the recommended path for scripted set-pieces [22]; (2) keep live-fracture piece counts low (30–50 pieces per breakable for gameplay cover); (3) use **Sleep/Disable Fields** to retire debris below a velocity threshold, cutting active rigid-body count [24]; (4) reserve destruction for *featured* objects, not every prop.

For *Operation Code Rescue*: convert a small library of cover types (crates, drywall panels, the destructible-tagged props that today are only tagged, not simulated) into Geometry Collections with ~20-piece uniform fracture and a fixed seed. Gate destruction triggers on the new `Cover` channel and the radial-impulse system from §6.3 so a grenade can blow open a barricaded doorway — a strong, ownable mechanic that also rewards spending scrap.

0.8 8. Vehicles: From `UFloatingPawnMovement` to Chaos for the Jeep

`AJeepActor` is currently an `APawn` driven by `UFloatingPawnMovement` with a `"Pawn"` collision box and a documented design intent of "no full PhysX wheel sim … good enough for fast traversal." That is a fine prototype, but it does not feel like a vehicle — no suspension, no weight transfer, no terrain response.

The commercial path is the **Chaos Vehicles** plugin and `UChaosWheeledVehicleMovementComponent` [25][26]. Setup requires a skeletal-mesh vehicle, a Physics Asset, an Animation Blueprint (parent `VehicleAnimationInstance`, using the **Wheel Controller** node), a Vehicle Blueprint (parent `WheeledVehiclePawn`), one or more **Wheel Blueprints** (parent `ChaosVehicleWheel`), and a **Float Curve** for the engine torque (X = RPM, Y = torque in Nm) [25]. The sequence [25]:

1. Enable **ChaosVehiclesPlugin** (Settings > Plugins > Physics; restart; PhysX must be off — UE 5.7 is already Chaos-only).
2. Build the Physics Asset: a Single Convex Hull body for the chassis, sphere bodies for wheels, suspension bones set to No Collision.
3. Create Wheel Blueprints; set **Wheel Radius** to match the art, **Affected by Engine** (rear = RWD), **Affected by Steering / Handbrake**, and **Max Steer Angle**.
4. In the movement component's **Wheel Setups**, map each Wheel Class to a bone (FL, FR, BL, BR).
5. Assign the torque curve under **Mechanical Setup > Engine Setup**.

Suspension and arcade-vs-sim tuning. The runtime API exposes exactly the knobs you need: `SetSuspensionParams(Rate, Damping, Preload, MaxRaise, MaxDrop, WheelIndex)`, plus `SetMaxEngineTorque`, `SetDriveTorque`, `SetDownforceCoefficient`, `SetDragCoefficient`, `SetWheelFrictionMultiplier`, and toggles like `SetABSEnabled` / `SetTractionControlEnabled`

[26]. For the Jeep's traversal role, tune toward **arcade**: high wheel friction multiplier (forgiving grip), generous suspension travel (MaxRaise/MaxDrop) to soak up rubble, moderate damping to avoid bounce, and a flat torque curve so acceleration feels responsive rather than realistic. Per-wheel `FWheelStatus` even reports the contacting `UPhysicalMaterial`, so the Jeep can kick up surface-appropriate dust [26].

macOS caveat. Chaos Vehicles is reliable on Metal but is sensitive to substep configuration — vehicle jitter on slopes is a known issue addressed by enabling substepping and tuning suspension spring/damping [3]. This is another reason to commit to the §2.2 substepping settings before polishing the Jeep. Keep the simple `UFloatingPawnMovement` Jeep as a fallback pawn behind a config flag while the Chaos vehicle is validated on the developer's hardware.

0.9 9. Cloth and Miscellaneous Simulation

Chaos Cloth can add motion to zombie rags, coats, and the rescue-point flags that mark helipads. The modern workflow is the **Panel Cloth Editor** (UE 5.3+), authoring a standalone **cloth asset** through a non-destructive **Dataflow graph**, applied at runtime via the **Chaos Cloth Component** on a skeletal mesh [27]. The panel and legacy editors share the same Chaos solver; the panel path adds optional XPBD constraints and level-set collision [27]. Cost is mostly per-component overhead (extra draw calls), so cloth is strictly **optional polish** [27]: reserve it for a few hero elements (a tattered flag at each helipad, a coat on a named survivor) rather than every zombie. For a solo developer, this is a Phase-4 "nice to have," not a launch requirement.

0.10 10. Maintaining Physics Quality

0.10.1 10.1 Determinism and frame-rate independence

The discipline established in §2 is what *keeps* physics correct as content scales: substepping for frame-rate-independent stability, fixed-step async only where reproducibility is required, and fixed fracture seeds so destruction is repeatable. Apply impulses in real-world units (kg, cm) consistently so tuning transfers between props.

0.10.2 10.2 Debugging tools

- `p.chaos.debugdraw.enabled 1` is the master gate for all Chaos physics-thread debug rendering — collision shapes, contacts, constraints — and must be on before most visualizers appear [28].
- The **Chaos Visual Debugger (CVD)** records physics-thread state during gameplay (locally or on a connected build) and lets you scrub frame-by-frame and per-substep, inspecting particle states, collisions, joints, and scene queries [29]. CVD 1.2 added a standalone debugger and session discovery for one-click recording [29]. Open it from **Tools > Debug > Chaos**

Visual Debugger. Note CVD shows *physics-thread* particle data, not game-thread data [30].

- `p.Chaos.ThreadingModel` switches the solver threading model at runtime for A/B testing [2].

0.10.3 10.3 Performance budgets on Apple Silicon

On a Metal/Apple-Silicon target, physics shares a thermal and bandwidth budget with the project's Lumen-software and virtual-shadow render path (already configured in `DefaultEngine.ini`). Practical budgets:

- **Simultaneous active ragdolls:** cap at ~8–12; freeze or fade older corpses (ties directly to the ragdoll work in §5.2).
- **Live-fracture pieces on screen:** ~150 total; prefer cached destruction for anything larger (§7).
- **Simulated loose props:** ensure damping-driven sleep so idle debris costs nothing (§4.4).
- Profile with `stat physics`, `stat chaos`, and CVD captures during the busiest horde encounters, on the actual target Mac — not just in PIE on a desktop.

0.10.4 10.4 Validation hooks (tie into existing project infrastructure)

The project already ships a `Smoke_Test_Packaged_App.command` and Python validators (`Scripts/verify_curriculum_validator_shapes.py`, `Scripts/mcp_fab_unreal_import_validate.py`). Extend that culture to physics:

1. **UE Data Validation.** Implement `IsDataValid` / `EDataValidationResult` checks on physics-bearing assets and actors: every zombie skeletal mesh has a Physics Asset; every breakable has a valid Geometry Collection with a fixed seed; the Jeep's Wheel Setups reference real bones. Data Validation surfaces these failures in the editor and in CI before they ship.
2. **Smoke-test extensions.** Add an automated pass to the packaged smoke test that spawns one of each physics actor (throwable, barricade, a ragdoll death, the Jeep) in an empty level and asserts no NaN transforms, no runaway velocities, and stable frame time over N seconds.
3. **Determinism check.** With a fixed fracture seed and async fixed-step enabled, record a short CVD capture of a scripted destruction and diff key body positions across runs to catch regressions.

0.10.5 10.5 A phased roadmap for a solo developer

Scope is everything for one person. Tackle physics in this order, each phase independently shippable:

Phase 1 — Foundation & Combat Juice (highest ratio of feel-per-hour). Define the custom collision channels (§3.3). Turn on substepping (§2.2). Add launch impulses + a physical-material impact system to existing systems: give `AThrowableActor` a real throw arc, migrate hitscan to the `Weapon` channel, and key impact VFX/SFX off `Hit.PhysMaterial`. No new assets required; immediate payoff.

Phase 2 — Reactive Enemies (the horror headline). Author/verify Physics Assets on the zombie meshes. Add ragdoll-on-death (§5.2) with an active-ragdoll budget, then layer the Physical Animation Component for hit flinches (§5.3) and wire bullet/explosion impulses into it.

Phase 3 — Destructible World. Convert a small cover library to Geometry Collections (§7) with fixed seeds and the Cache System for set-pieces; connect the radial-impulse explosion system so grenades open barricaded doorways.

Phase 4 — Vehicle & Cloth Polish. Upgrade the Jeep to Chaos Vehicles with arcade tuning (§8), keeping the `UFloatingPawnMovement` version behind a fallback flag until validated on-device. Optionally add hero cloth to flags and named survivors (§9).

Throughout — Maintenance. Add the Data Validation rules and smoke-test extensions of §10.4 *as each phase lands*, so the physics that works today still works after the 465th city is built.

0.11 References

[1] Physics in Unreal Engine — https://dev.epicgames.com/documentation/en-us/unreal-engine/physics-in-unreal-engine [2] Physics Settings in the Unreal Engine Project Settings — https://dev.epicgames.com/documentation/en-us/unreal-engine/physics-settings-in-the-unreal-engine-project-settings [3] Unreal Engine 5.7 Chaos Vehicle Jitter on Slopes — Substepping and Suspension Tuning Fix — https://gamineai.com/help/unreal-engine-5-7-chaos-vehicle-jitter-slopes-substepping-suspension-fix [4] Physics Sub-Stepping in Unreal Engine — https://dev.epicgames.com/documentation/en-us/unreal-engine/physics-sub-stepping-in-unreal-engine [5] Event Async Physics Tick (Blueprint API) — https://dev.epicgames.com/documentation/en-us/unreal-engine/BlueprintAPI/AddEvent/EventAsyncPhysicsTick [6] Collision Response Reference in Unreal Engine — https://dev.epicgames.com/documentation/en-us/unreal-engine/collision-response-reference-in-unreal-engine [7] Creating Custom Collision Channels and Profiles (UhiyamaLab) — https://uhiyama-lab.com/en/notes/ue/collision-profile-custom-channel/ [8] Hybrid Collision/Trace Interact System Using Custom Object and Trace Channels in Unreal Engine (Brian Stong) — https://medium.com/@stonger44/hybrid-collision-trace-interact-system-using-custom-object-and-trace-channels-in-unreal-engine-b94952eeb0e6 [9] Physical Materials User Guide for Unreal Engine — https://dev.epicgames.com/documentation/en-us/unreal-engine/physical-materials-user-guide-for-unreal-engine [10] Physics Damping in Unreal Engine — https://dev.epicgames.com/documentation/en-us/unreal-engine/physics-damping-in-unreal-engine [11] Physics Constraints in Unreal Engine — https://dev.epicgames.com/documentation/en-us/unreal-engine/physics-constraints-in-unreal-engine [12] Movement Components in Unreal Engine — https://dev.epicgames.com/documentation/en-us/unreal-engine/movement-components-in-unreal-engine [13] Physics Asset Editor in Unreal Engine — https://dev.epicgames.com/documentation/en-us/unreal-engine/physics-asset-editor-in-unreal-engine [14] Physics-Based Animation in Unreal Engine — https://dev.epicgames.com/documentation/en-us/unreal-engine/physics-driven-animation-in-unreal-engine [15] Applying a Physical Animation Profile in Unreal Engine — https://dev.epicgames.com/documentation/en-us/unreal-engine/applying-a-physical-animation-profile-in-unreal-engine [16] Applying a Physical Animation Profile in Unreal

Engine (Physical Animation Component workflow) — https://dev.epicgames.com/documentation/en-us/unreal-engine/applying-a-physical-animation-profile-in-unreal-engine [17] Implement a Projectile in Unreal Engine (AddImpulseAtLocation) — https://dev.epicgames.com/documentation/en-us/unreal-engine/coder-08-implement-a-projectile-in-unreal-engine [18] Implement a Projectile in Unreal Engine (hitscan line trace + projectile defaults) — https://dev.epicgames.com/documentation/en-us/unreal-engine/coder-08-implement-a-projectile-in-unreal-engine [19] UProjectileMovementComponent (C++ API Reference) — https://dev.epicgames.com/documentation/en-us/unreal-engine/API/Runtime/Engine/UProjectileMovementComponent [20] Add Radial Impulse (Blueprint API) — https://dev.epicgames.com/documentation/en-us/unreal-engine/BlueprintAPI/Physics/AddRadialImpulse [21] Physical Materials Reference for Unreal Engine — https://dev.epicgames.com/documentation/en-us/unreal-engine/physical-materials-reference-for-unreal-engine [22] Destruction Overview — https://dev.epicgames.com/documentation/en-us/unreal-engine/destruction-overview [23] Fracturing Geometry Collections User Guide — https://dev.epicgames.com/documentation/unreal-engine/fracturing-geometry-collections-user-guide [24] Chaos Fields User Guide in Unreal Engine — https://dev.epicgames.com/documentation/en-us/unreal-engine/chaos-fields-user-guide-in-unreal-engine [25] How to Set up Vehicles in Unreal Engine — https://dev.epicgames.com/documentation/en-us/unreal-engine/how-to-set-up-vehicles-in-unreal-engine [26] UChaosWheeledVehicleMovementComponent (C++ API Reference) — https://dev.epicgames.com/documentation/en-us/unreal-engine/API/Plugins/ChaosVehicles/UChaosWheeledVehicleMovementComp- [27] Panel Cloth Editor Overview — https://dev.epicgames.com/documentation/unreal-engine/panel-cloth-editor-overview [28] Vehicle Debug Commands in Unreal Engine — https://dev.epicgames.com/documentation/en-us/unreal-engine/vehicle-debug-commands-in-unreal-engine [29] Chaos Visual Debugger in Unreal Engine — https://dev.epicgames.com/documentation/en-us/unreal-engine/chaos-visual-debugger-in-unreal-engine [30] Data Visualization Flags in Chaos Visual Debugger — https://dev.epicgames.com/documentation/en-us/unreal-engine/data-visualization-flags-in-chaos-visual-debugger